

When better candidates hurt search: paired counterfactual audits of candidate replacement in differential evolution, particle swarms and CMA-ES

Short title: Candidate replacement and later search outcomes

Alan Vallavaraj^{1*}

¹ School of Computer Science and Engineering, University of Westminster, London, United Kingdom

* Corresponding author: a.vallavaraj@westminster.ac.uk

Abstract

Candidate-interception methods can substitute an optimizer’s next proposed point without increasing that optimizer’s online evaluation budget. I ask whether a replacement with a lower immediate objective value improves the optimizer after an equal continuation budget. At each checkpoint I copy the complete state of differential evolution (DE), particle swarm optimization (PSO), or covariance matrix adaptation evolution strategy (CMA-ES), evaluate the original or transformed point in separate branches, and then continue each branch without intervention under matched random streams. Across 24 noiseless COCO/BBOB functions, dimensions 5, 10 and 20, two instances and two seeds per dimension, contraction gave an immediately better candidate in 267 DE, 257 PSO and 240 CMA-ES forks. Despite that improvement, the transformed branch later had worse best-so-far fitness in 48/267, 26/257 and 134/240 forks, respectively. On unused BBOB instance IDs, a 576-checkpoint CMA-ES audit found delayed harm after 208/478 immediately better candidates under direct replacement and 211/478 under the documented `inject()` interface. Injecting was better than direct replacement in 277 continuations, worse in 273 and tied in 26. The diagnostic forks themselves require additional real evaluations. These findings establish a failure mode for immediate candidate-quality proxies in the tested settings, and motivate trajectory-aware validation of candidate-interception methods; they do not identify an optimizer that consistently outperforms published methods.

Introduction

Surrogate-assisted optimization often proposes candidates before expensive evaluation. Classical response-surface and infill approaches include efficient global optimization, its taxonomy, online approximations and multiobjective extensions [1–4]. Approximate fitness, surrogate-assisted evolutionary computation and surveys examine the risks of prediction error and limited evaluation budgets [5–8]. The surrogate may filter or replace proposed points and alter the future algorithm state. Generic wrappers and probabilistic surrogate frameworks make such replacement possible across hosts [9–13]. Thus generic candidate interception itself is established.

Applications and alternative mechanisms span dynamic optimization, data-driven constrained systems, local/global metamodels, robust design and parameter control [14–18]. Evolution-strategy surrogates and metamodel-assisted preselection provide additional host-specific precedents [19, 20]. Differential evolution and swarm optimization admit classification-assisted replacement, cooperative search, active learning and multiple local surrogates [21–26]. Expensive many-objective work uses classification, decomposition, constraints and parallel infill; these are relevant to the scope of candidate selection, although our experiments have one objective and no constraints [27–33]. Widely used implementation platforms are described in [34, 35].

The hosts draw on the original DE, PSO and CMA-ES designs [36–38]. Hansen explicitly described external-solution injection and step renormalization in CMA-ES [39]. Contemporary work on budget-neutral geometry-guided infill and on delayed lineage value further narrows any originality claim concerning candidate modification or long-horizon credit [40, 41]. This study contributes an empirical measurement of how often an *actually better evaluated candidate* fails to improve the later trajectory in three specified host implementations, plus a fresh-instance comparison of direct CMA-ES replacement with native injection. This is an evaluation study rather than a new optimization algorithm.

Materials and methods

Paired state-clone contrast

For minimization, let S_t denote the complete host state and x_t its next proposal. A treatment transforms x_t into $T(x_t, S_t)$; PASS evaluates the original. I clone the host and its random state before proposal, evaluate exactly one point in each branch at the intervention step, and then run $H - 1$ ordinary host evaluations in each branch. Both branches therefore have the same continuation budget, while producing both branches for diagnosis costs additional true evaluations. Define

$$\delta_1 = f(x_t) - f(T(x_t, S_t)), \quad \Delta_H = b_{t+H}^{\text{PASS}} - b_{t+H}^T, \quad (1)$$

where b is best-so-far objective value. Conditional harm is $\Delta_H < -10^{-9}$ among checkpoints with $\delta_1 > 10^{-9}$. This compares the *candidate* immediately, not necessarily a new incumbent. PASS candidate fitness is observed only in the diagnostic experiment and is not available for free to a live decision policy. Common random numbers improve precision without forcing the subsequently diverged trajectories to have identical proposals.

Two fixed transformations are studied: contraction $T_C(x) = \text{clip}\{x + 0.45(x_{\text{best}} - x)\}$ and step reflection $T_R(x) = \text{clip}\{x_{\text{parent}} - 0.75(x - x_{\text{parent}})\}$. The clip uses the problem bounds. DE uses $\text{rand}/1/\text{bin}$, $F = 0.7$, crossover probability 0.9 and elitist parent replacement. PSO uses inertia 0.7, cognitive and social coefficients 1.4 and a global best. The PSO treatment changes the particle’s position and velocity. CMA-ES uses `cma` 4.5.0 ask/tell, center initialization, an initial step size 0.3 times average box width and population $\max(8, \lfloor 4 + 3 \log D \rfloor)$; the other hosts use $\max(8, 5D)$ individuals. These are defined host implementations rather than claims about all DE, PSO and CMA-ES variants.

Implementation and paired random streams

The released `Host` object contains population vectors and fitnesses, the best observed value, an evaluation counter, and a NumPy random generator initialized with seed 0 or 1 for the main experiment. DE samples three distinct donors other than the parent and forces at least one crossover coordinate. PSO keeps each

particle’s personal best and velocity; when a substituted position is accepted, its velocity is reset to the displacement from its preceding position. The CMA-ES state includes the `CMAEvolutionStrategy` instance, its current generation and the NumPy random state used during `ask()`; its separate CMA-ES seed is the experiment seed plus one. CMA-ES receives a whole generation in `tell()`, even when the intervention changes one member of that generation. In the main study the CMA-ES checkpoint is after exactly $20D$ evaluations, so the first proposal is part of the ensuing generation. Each problem starts a fresh host from its specified seed.

Python’s `deepcopy` duplicates this state immediately *before* calling `propose()`; only the deterministic COCO problem object is shared. PASS, contraction and reflection therefore start with identical adaptive parameters and random-generator states. The intervention occurs on the first proposal only; all subsequent proposals use the ordinary host update. The same random draws are available across branches initially, but differing state updates can turn later draws into different candidate positions. The recorded best-so-far values are taken after evaluations 1, 10, 50 and H of each continuation. For each fork, the baseline fitness, PASS fitness, intervention fitness and later best-so-far values can be inspected directly in the released CSV. A byte-for-byte replay of 60 PASS objective values on a separate test state checks deterministic copying for each host; a one-problem rerun of the released protocol checks both the main and injection CSV outputs.

Benchmark and evaluation accounting

COCO’s noiseless BBOB suite comprises 24 functions; multiple instances change underlying shifts and transformations [42, 43]. I use all 24 functions at $D \in \{5, 10, 20\}$, two instances and two seeds per dimension. Main experiment suite indices 6–7, 8–9 and 12–13 correspond to actual instance IDs 71–72, 73–74 and 77–78. Each baseline uses $20D$ true evaluations; each of PASS, contraction and reflection uses a further $20D$. This gives 288 checkpoint states per dimension, 864 branch rows per dimension, and 806,400 true evaluations over the three dimensions. A regression check copies PASS twice and verifies identical first and subsequent objectives on a test problem.

The cap check has separate instance IDs 75–76, two seeds and 96 checkpoints at each of 5D and 10D. The displacement from an original proposal to its contracted proposal is capped at $0.5\sqrt{D}$ Mahalanobis units under the current CMA-ES search distribution. This is a specific fixed cap, not a replication of the entire injection method of [39]. PASS, contraction and capped contraction receive identical $20D$ branch budgets. The two checks add 115,200 true evaluations.

An additional CMA-ES audit uses unused instance IDs 79–80, four seeds, all 24 functions and three dimensions. Checkpoints occur after 104, 200 and 408 evaluations respectively, at complete generation boundaries. Each state produces PASS, direct replacement in `tell()`, and `CMAEvolutionStrategy.inject([genotype], force=True)` before `ask()`. The bounded candidate is converted back to genotype with the library boundary inverse. The two treated branches evaluate the same first candidate within floating-point tolerance; native injection can change subsequent assignment of common random variates to samples. The continuation is $20D$ evaluations without further treatment. The 576 checkpoints yield 1,728 branch rows and 539,904 true calls including baseline and branches.

Function-cluster bootstrap intervals resample the 24 functions, preserving the within-function dependence of instance and seed repetitions. These are descriptive uncertainty intervals for the specified benchmark design, not coverage guarantees for unseen real-world functions. All counts and interval computations are regenerated by the released scripts.

Each checkpoint supplies a paired PASS–treatment contrast. A difference of magnitude at most 10^{-9} is a tie for both candidate and later-best comparisons. Conditional harm denominators include only forks whose treated first candidate is better than PASS by more than 10^{-9} . The dimension-specific host comparison first computes the difference between CMA-ES and DE conditional harm rates within each BBOB function and then averages across functions with eligible forks in both hosts. Its percentile interval uses 4,000 bootstrap resamples of functions with replacement (NumPy seed 390). For the injection study, direct and injected conditional-harm indicators are paired within checkpoint; the equal-function mean difference and percentile interval use 5,000 function bootstrap draws (seed 1701). This resampling treats the 24 functions as clusters rather than treating correlated instances and seeds as independent. Reported intervals and pooled counts are descriptive: no family-wise testing or claims of population-wide coverage are made.

Use of artificial intelligence tools

OpenAI ChatGPT (Codex) assisted with research planning, Python implementation, experiment execution support, figure generation, literature lookup, and drafting and formatting this manuscript. Numerical results come from executed Python experiments recorded in branch-level CSV files, not language-model estimates. The generated code and outputs were checked by deterministic clone replay, structural checks of the released branch records, re-execution of representative experiment checkpoints, and re-running the released analysis scripts to reproduce their saved summaries. These checks do not constitute independent replication of the entire experiment by a separate implementation. The author is responsible for reviewing the code, methods, source citations, interpretation, and final submission.

Results

Contraction and reflection

Table 1 summarizes contraction outcomes by dimension and host. Fig 1 shows conditional harm rates with

Table 1. Harmful later best-so-far outcome conditional on an immediately better contraction candidate. Fractions show harm / immediately better candidates; hosts need not visit the same states.

Dimension	DE	PSO	CMA-ES
5	18/87 (20.7%)	8/77 (10.4%)	41/80 (51.3%)
10	18/89 (20.2%)	10/88 (11.4%)	47/79 (59.5%)
20	12/91 (13.2%)	8/92 (8.7%)	46/81 (56.8%)
Pooled descriptive	48/267 (18.0%)	26/257 (10.1%)	134/240 (55.8%)

uncertainty intervals resampled by BBOB function. Reflection gives fewer immediately better proposals:

Fig 1. Conditional contraction harm by dimension. Intervals resample the 24 benchmark functions as clusters; labels above the marks give counts of harmful later outcomes over immediately better first candidates.

descriptive pooled conditional harm is 7/108 in DE, 1/45 in PSO and 59/104 in CMA-ES. The function-cluster mean harm-rate difference (CMA-ES minus DE) for contraction is 0.281 (95% bootstrap interval 0.170–0.389) in 5D, 0.410 (0.246–0.562) in 10D, and 0.448 (0.340–0.552) in 20D. These are between-host associations under different visited states, populations and proposals; they do not isolate the covariance update as the cause.

The fixed cap beats/loses/ties against uncapped contraction in 14/17/65 five-dimensional forks and 2/6/88 ten-dimensional forks. It changes the first candidate’s value in only 31/96 and 8/96 forks respectively. This cap was often inactive and did not resolve the observed harm; stronger conclusions about injection safeguards do not follow.

Fresh-instance injection audit

Table 2 reports outcomes on unused instance IDs 79–80. Fig 2 compares delayed harm rates for direct replacement and native injection. The equal-function mean difference in conditional harm (direct minus

Fig 2. Delayed harm after direct replacement and native injection. The two treatments share the same first contracted candidate within numerical tolerance. The vertical intervals resample BBOB functions as clusters; instances and seeds remain grouped within function.

injected), pooled descriptively over dimensions, is -0.002 with function-cluster bootstrap interval $[-0.054, 0.053]$. Injection wins/loses/ties against direct treatment in 277/273/26 checkpoints. This evidence

Table 2. Fresh-instance CMA-ES audit. Harm is conditional on the direct and injected branches sharing an immediately better first candidate.

D	Checkpoints	Better first	Direct harm	Inject harm
5	192	161	74/161 (46.0%)	65/161 (40.4%)
10	192	153	63/153 (41.2%)	66/153 (43.1%)
20	192	164	71/164 (43.3%)	80/164 (48.8%)
Pooled	576	478	208/478 (43.5%)	211/478 (44.1%)

does not show consistent rescue by the native interface under the tested contraction source. The documented interface may still protect against different external candidates or instabilities. Recorded mean shift, step-size ratio and covariance ratios after the first generation do not isolate one decisive mechanism; direct and injected branches also differ in later proposal assignment.

Discussion

The within-host PASS-versus-treatment branch difference is a causal contrast at each cloned checkpoint for the specified one-time transformation and continuation. The conditional harm rate is selected by the observed immediate candidate difference; it is a diagnostic rate and does not evaluate a deployable gate. Host comparisons are not randomized across identical state distributions. Only deterministic, box-constrained, noiseless benchmark problems, two transformations, basic hosts and one continuation horizon were evaluated. Fresh injection instances test a new indexing and seed regime on the *same* 24 function definitions, not a wholly independent function suite. Diagnostic computation is much larger than the online budget of one branch. Neither a deployable improvement policy nor state-of-the-art optimization performance has been demonstrated.

Several algorithmic mechanisms could produce a lower first fitness followed by a worse continuation. In DE, replacing a trial changes whether its parent survives and which point can later become a donor. In PSO, a replacement changes a particle’s position and velocity and can update personal or global best; these state changes influence subsequent proposals. In CMA-ES, a changed member of a generation can alter fitness ranking and hence adaptation of the mean, covariance and step size. Such effects can be favorable or unfavorable depending on the local landscape and remaining budget. These mechanisms follow from the specified update rules, but the present between-host counts do not establish their relative causal importance. In the injection audit, changing proposal assignment after `inject()` further complicates any attribution to a single adaptation variable.

Earlier surrogate-assisted work mainly asks whether a model can reduce expensive evaluations or improve the optimizer’s realized performance [6, 12, 13, 20]; native CMA-ES injection directly addresses handling external points [39]. Here a deliberately simple deterministic transform removes surrogate prediction error from the immediate comparison. The observed reversals therefore illustrate why a candidate’s measured objective value is an incomplete proxy for the value of the resulting search state. This complements, but does not benchmark against, those surrogate methods or the different long-horizon search settings of [41].

A submission claiming practical optimizer improvement would require independently implemented hosts, stronger new problem distributions, more configurations and noise, target-based and anytime COCO metrics, measured online budgets, and comparisons against established surrogate and injection methods. Controlled ablations would be required to distinguish covariance adaptation from different proposed points and other host-state effects. The current data support a narrower contribution: immediately better candidate quality is an unreliable proxy for later trajectory value in these specified conditions. Branch-level outcomes at the recorded horizons, fixed scripts and instructions accompany the manuscript for independent checking.

Conclusions

The paired forks demonstrate that a lower objective value for one transformed candidate does not reliably imply a better later best-so-far value in the studied host states. This failure mode persists on separately indexed BBOB instances for CMA-ES under both direct replacement and documented injection. These results motivate evaluating candidate-interception layers using trajectory outcomes and complete evaluation accounting rather than immediate candidate quality alone; broader generalization and a corrective online policy require further experiments.

Data availability

The branch-level values at the recorded horizons, identifiers, analysis code, figure source scripts, and reproduction instructions are publicly available at <https://github.com/alanvallavaraj/candidate-intervention-trajectory-audit>. The study uses public noiseless BBOB functions implemented by COCO and generates no human-subject data.

Acknowledgments

The author thanks the maintainers of the open-source COCO/BBOB and pycma software used for this study.

References

1. Jones DR, Schonlau M, Welch WJ. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*. 1998;13(4):455-92. doi:10.1023/A:1008306431147.
2. Jones DR. A taxonomy of global optimization methods based on response surfaces. *Journal of Global Optimization*. 2001;21(4):345-83. doi:10.1023/A:1012771025575.
3. Knowles J. ParEGO: a hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE Transactions on Evolutionary Computation*. 2006;10(1):50-66. doi:10.1109/TEVC.2005.851274.
4. Zhang Q, Liu W, Tsang E, Virginas B. Expensive multiobjective optimization by MOEA/D with gaussian process model. *IEEE Transactions on Evolutionary Computation*. 2010;14(3):456-74. doi:10.1109/TEVC.2009.2033671.
5. Jin Y, Olhofer M, Sendhoff B. A framework for evolutionary optimization with approximate fitness functions. *IEEE Transactions on Evolutionary Computation*. 2002;6(5):481-94.
6. Jin Y. Surrogate-assisted evolutionary computation: Recent advances and future challenges. *Swarm and Evolutionary Computation*. 2011;1(2):61-70. doi:10.1016/j.swevo.2011.05.001.
7. Haftka RT, Villanueva D, Chaudhuri A. Parallel surrogate-assisted global optimization with expensive functions: a survey. *Structural and Multidisciplinary Optimization*. 2016;54(1):3-13. doi:10.1007/s00158-016-1432-3.
8. Stork J, Friese M, Zaefferer M, Bartz-Beielstein T, Fischbach A, Breiderhoff B, et al. Open issues in surrogate-assisted optimization. In: *High-Performance Simulation-Based Optimization*. Springer; 2020. p. 225-44.
9. Ong YS, Nair PB, Keane AJ. Evolutionary optimization of computationally expensive problems via surrogate modeling. *AIAA Journal*. 2003;41(4):687-96. doi:10.2514/2.1999.
10. Lim D, Jin Y, Ong Y, Sendhoff B. Generalizing surrogate-assisted evolutionary computation. *IEEE Transactions on Evolutionary Computation*. 2010;14(3):329-55.

11. Cai X, Gao L, Li X. Efficient generalized surrogate-assisted evolutionary algorithm for high-dimensional expensive problems. *IEEE Transactions on Evolutionary Computation*. 2020;24(2):365-79. doi:10.1109/TEVC.2019.2919762.
12. Blank J, Deb K. PSAF: A Probabilistic Surrogate-Assisted Framework for Single-Objective Optimization. In: *Proceedings of the Genetic and Evolutionary Computation Conference*; 2021. p. 652-9. doi:10.1145/3449639.3459297.
13. Blank J, Deb K. GPSAF: A Generalized Probabilistic Surrogate-Assisted Framework for Constrained Single- and Multi-objective Optimization. *arXiv preprint*. 2022. ArXiv:2204.04054. Available from: <https://arxiv.org/abs/2204.04054>.
14. Luo W, Yi R, Yang B, Xu P. Surrogate-assisted evolutionary framework for data-driven dynamic optimization. *IEEE Transactions on Emerging Topics in Computational Intelligence*. 2019;3(2):137-50. doi:10.1109/TETCI.2018.2872029.
15. Wang H, Jin Y. A random forest-assisted evolutionary algorithm for data-driven constrained multiobjective combinatorial optimization of trauma systems. *IEEE Transactions on Cybernetics*. 2020;50(2):536-49. doi:10.1109/TCYB.2018.2869674.
16. Zhou Z, Ong YS, Nair PB, Keane AJ, Lum KY. Combining global and local surrogate models to accelerate evolutionary optimization. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*. 2007;37(1):66-76. doi:10.1109/TSMCC.2005.855506.
17. Ong YS, Nair PB, Lum KY. Max-min surrogate-assisted evolutionary algorithm for robust design. *IEEE Transactions on Evolutionary Computation*. 2006;10(4):392-404.
18. Bagheri S, Konen W, Emmerich M, Bäck T. Self-adjusting parameter control for surrogate-assisted constrained optimization under limited budgets. *Applied Soft Computing*. 2017;61:377-93.
19. Emmerich M, Giotis A, Özdemir M, Bäck T, Giannakoglou K. Metamodel-assisted evolution strategies. In: *Parallel Problem Solving from Nature VII*; 2002. p. 361-70.
20. Hansen N. A global surrogate assisted CMA-ES. In: *Proceedings of GECCO*; 2019. p. 664-72. doi:10.1145/3321707.3321842.
21. Lu X, Tang K, Yao X. Classification-assisted Differential Evolution for computationally expensive problems. In: *IEEE Congress on Evolutionary Computation*; 2011. p. 1986-93. doi:10.1109/CEC.2011.5949859.
22. Wang Y, Yin D, Yang S, Sun G. Global and local surrogate-assisted differential evolution for expensive constrained optimization problems with inequality constraints. *IEEE Transactions on Cybernetics*. 2019;49(5):1642-56.
23. Sun C, Jin Y, Cheng R, Ding J, Zeng J. Surrogate-assisted cooperative swarm optimization of high-dimensional expensive problems. *IEEE Transactions on Evolutionary Computation*. 2017;21(4):644-60.
24. Lv Z, Wang L, Han Z, Zhao J, Wang W. Surrogate-assisted particle swarm optimization algorithm with Pareto active learning for expensive multi-objective optimization. *IEEE/CAA Journal of Automatica Sinica*. 2019;6(3):838-49. doi:10.1109/JAS.2019.1911450.
25. Wang H, Jin Y, Doherty J. Committee-based active learning for surrogate-assisted particle swarm optimization of expensive problems. *IEEE Transactions on Cybernetics*. 2017;47(9):2664-77.
26. Li F, Cai X, Gao L, Shen W. A surrogate-assisted multiswarm optimization algorithm for high-dimensional computationally expensive problems. *IEEE Transactions on Cybernetics*. 2021;51(3):1390-402. doi:10.1109/TCYB.2020.2967553.

27. Habib A, Singh HK, Chugh T, Ray T, Miettinen K. A multiple surrogate assisted decomposition-based evolutionary algorithm for expensive multi/many-objective optimization. *IEEE Transactions on Evolutionary Computation*. 2019;23(6):1000-14. doi:10.1109/TEVC.2019.2899030.
28. Pan L, He C, Tian Y, Wang H, Zhang X, Jin Y. A classification-based surrogate-assisted evolutionary algorithm for expensive many-objective optimization. *IEEE Transactions on Evolutionary Computation*. 2019;23(1):74-88. doi:10.1109/TEVC.2018.2802784.
29. Deb K, Hussein R, Roy PC, Toscano-Pulido G. A taxonomy for metamodeling frameworks for evolutionary multiobjective optimization. *IEEE Transactions on Evolutionary Computation*. 2019;23(1):104-16.
30. Viana F, Haftka R. Surrogate-based optimization with parallel simulations using the probability of improvement. In: 13th AIAA/ISSMO Multidisciplinary Analysis Optimization Conference; 2010. p. 9392. doi:10.2514/6.2010-9392.
31. Beaucaire P, Beauthier C, Sainvitu C. Multi-point infill sampling strategies exploiting multiple surrogate models. In: GECCO Companion; 2019. p. 1559-67. doi:10.1145/3319619.3328527.
32. Berveglieri N, Derbel B, Liefoghe A, Aguirre H, Zhang Q, Tanaka K. Designing parallelism in surrogate-assisted multiobjective optimization based on decomposition. In: Proceedings of GECCO; 2020. p. 462-70. doi:10.1145/3377930.3390202.
33. Bagheri S, Konen W, Allmendinger R, Branke J, Deb K, Fieldsend J, et al. Constraint handling in efficient global optimization. In: Proceedings of GECCO; 2017. p. 673-80. doi:10.1145/3071178.3071278.
34. Blank J, Deb K. pymoo: Multi-objective Optimization in Python. *IEEE Access*. 2020;8:89497-509. doi:10.1109/ACCESS.2020.2990567.
35. Tian Y, Cheng R, Zhang X, Jin Y. PlatEMO: A MATLAB platform for evolutionary multi-objective optimization. *IEEE Computational Intelligence Magazine*. 2017;12(4):73-87. doi:10.1109/MCI.2017.2742868.
36. Storn R, Price K. Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*. 1997;11:341-59. doi:10.1023/A:1008202821328.
37. Kennedy J, Eberhart R. Particle swarm optimization. In: Proceedings of the IEEE International Conference on Neural Networks. vol. 4; 1995. p. 1942-8.
38. Hansen N, Ostermeier A. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*. 2001;9(2):159-95. doi:10.1162/106365601750190398.
39. Hansen N. Injecting External Solutions Into CMA-ES. arXiv preprint. 2011. ArXiv:1110.4181. Available from: <https://arxiv.org/abs/1110.4181>.
40. Yu H, Xu P, Yi J, Tian Y, Sun M, Lu J, et al. JANUS: Online Jacobian-Aligned Infill for Black-Box Optimization. arXiv preprint. 2026. ArXiv:2608.22862. Available from: <https://arxiv.org/abs/2608.22862>.
41. Siper M, Khalifa A, Togelius J. The Time Value of Evolution. arXiv preprint. 2026. ArXiv:2608.13297. Available from: <https://arxiv.org/abs/2608.13297>.
42. Hansen N, Finck S, Ros R, Auger A. Real-Parameter Black-Box Optimization Benchmarking 2009: Noiseless Functions Definitions. INRIA; 2009. RR-6829. Available from: <https://hal.science/inria-00362633>.
43. Hansen N, Auger A, Ros R, Mersmann O, Tušar T, Brockhoff D. COCO: A platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*. 2021;36(1):114-44. doi:10.1080/10556788.2020.1808977.